

nyrf OS: what each file does

The project/ tree after POC 1 was moved into it. Nothing here is new code: it is POC 1 rearranged into a layout that can grow.

The tree

```
project/
  Makefile
  boot/          main.c  elf.c  elf.h
  include/       uefi.h  bootinfo.h
  include/kernel/ serial.h
  kernel/        main.c  link.ld
  kernel/arch/x86_64/ entry.asm
  kernel/dev/    serial.c
  kernel/mm/     (empty)
  kernel/lib/    (empty)
```

Boot order

Which file is running, from power-on to the halt.

```
firmware -> boot/main.c      efi_main, talks to UEFI
          -> boot/elf.c      loads kernel.elf into memory
          -> ExitBootServices  firmware is gone from here on
          -> entry.asm        _start, first kernel instruction
          -> kernel/main.c    kernel_main, draws, halts
```

Build

Makefile

Compiles the tree twice. The bootloader as PE32+ with the Microsoft ABI, the kernel as ELF64 with System V. Also builds the disk image and the QEMU targets. Source lists are globbed, so a new .c or .asm anywhere in the tree is picked up without editing it.

Shared

Used by both halves of the system, or by the boundary between them.

include/uefi.h

The roughly sixty UEFI types and protocol structures the bootloader actually touches, transcribed from the UEFI 2.10 specification rather than pulling in gnu-efi. Bootloader only; the kernel never sees it.

include/bootinfo.h

boot_info_t, the single structure the bootloader hands to the kernel: framebuffer, memory map, ACPI RSDP. It is compiled under two different ABIs, so it is packed and carries _Static_assert on every field offset. A future edit that moves a field breaks the build instead of garbling the screen.

include/kernel/serial.h + kernel/dev/serial.c

16550 UART driver on COM1, polled, no interrupts. Linked into both halves because it is the only output channel that survives ExitBootServices. Without it, a black screen cannot be told apart from a kernel that was never reached.

Bootloader

Runs under the firmware, before the operating system exists.

boot/main.c

`efi_main`. Disarms the watchdog, finds the framebuffer through the Graphics Output Protocol, reads the memory map, finds the ACPI RSDP, opens `kernel.elf`, fills in `boot_info_t`, calls `ExitBootServices` and jumps to the kernel. Everything the firmware knows is collected here, because afterwards there is no one left to ask.

boot/elf.h + boot/elf.c

ELF64 loader. Validates the header, walks the program header table, allocates each `PT_LOAD` segment at its `p_paddr` and copies it in, then zeroes the gap between `p_filesz` and `p_memsz`, which is the kernel's `.bss`.

Kernel

Runs on bare metal. No firmware, no way back.

kernel/arch/x86_64/entry.asm

`_start`, the first instruction the kernel runs. Masks interrupts (there is no IDT yet, so none may fire), clears the direction flag, switches to a stack of its own rather than the firmware's, and calls into C with the `boot_info_t` pointer still in `RDI`.

kernel/link.ld

Memory layout. Pins the kernel at physical `0x100000` and places `.text.entry` first, so `_start` sits at exactly that address and is easy to check in GDB.

kernel/main.c

`kernel_main`. Checks the magic in `boot_info_t`, draws the RGB test pattern straight into the framebuffer, then halts forever.

kernel/mm/ kernel/lib/

Empty. The physical memory manager and the freestanding helpers go here.

What changes next

The GDT is the first new code. It touches `kernel/arch/x86_64/` and `entry.asm`. The bootloader half can be left alone for a long time: it has already done its job by the time any of the remaining work begins.